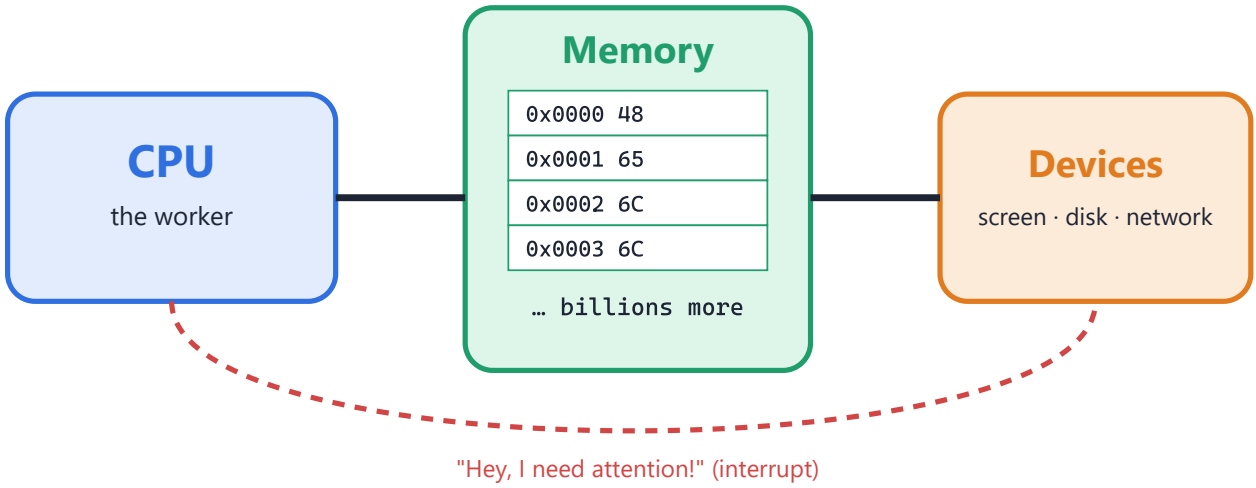
KVM-HELLO STUDY NOTES · LEVEL 0 OF 8

Computer basics: what the machine actually does

Before you can fake a computer (which is what a hypervisor does), you need to know what a real one does. This level covers the five ideas that every later level builds on.



0.1 CPU, memory and addresses

0.2 Registers: the CPU's pockets

0.3 Binary, hex, bits and flags

0.4 Talking to devices: I/O ports vs MMIO

0.5 Interrupts

✦ Putting it all together: `guests/hello16.asm` , line by line

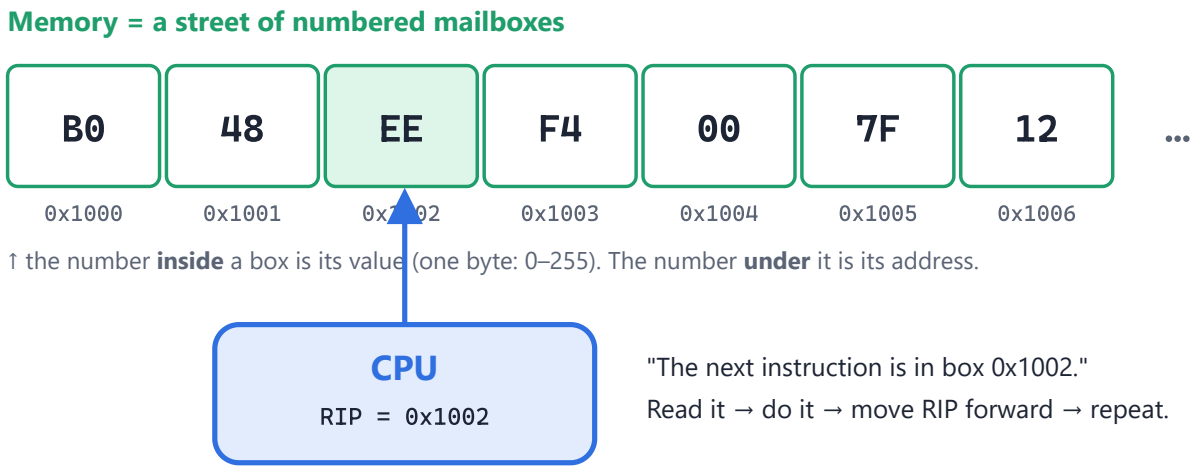
✦ Cheat sheet and answers

How to study: read the big picture first, then "Go deeper". Answer the "Check yourself" questions out loud before peeking at the answers at the back.

0.1

CPU, memory and addresses

> Memory is a **giant row of numbered boxes**. Each box holds one byte.
> The box's number is its **address**.
> The CPU is a worker who **reads an instruction from a box, does it, then moves to the next box**, forever.



In plain words

A computer has two main parts for this project. The **CPU** does the work and **memory (RAM)** holds the data. Everything the CPU knows how to do (add, copy, compare, jump) is encoded as bytes, and those bytes sit in memory right next to ordinary data. A program is just bytes in memory that the CPU happens to be pointed at.

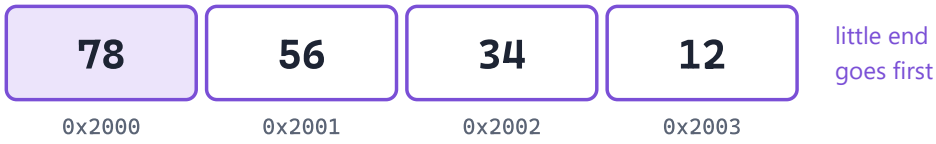
GO DEEPER

1. Byte-addressable memory

- The smallest addressable unit is a **byte** (8 bits, values 0–255). Address `0x1000` is one byte and `0x1001` is the next byte.
- Bigger values span several neighbouring bytes: a **word** is 2 bytes, a **dword** 4 and a **qword** 8. The CPU names a multi-byte value by its *lowest* address.
- Memory is **dumb**. It doesn't know whether a byte is code, text, a number or a pointer. Meaning comes only from *how the CPU uses it*.

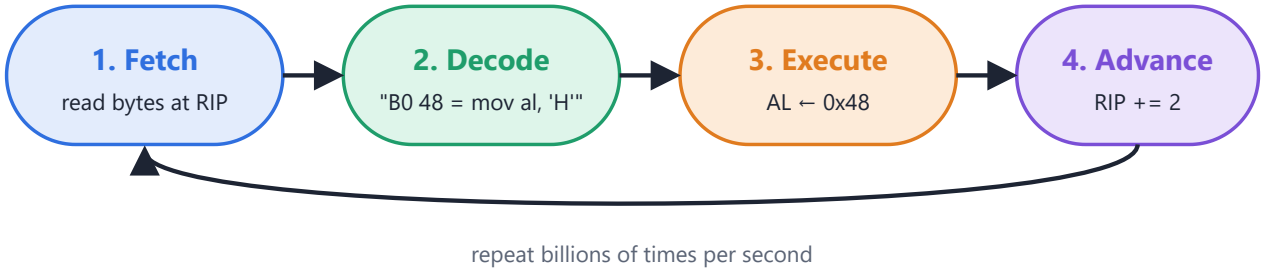
2. Little-endian: x86 stores numbers "backwards"

Store the 4-byte number **`0x12345678`** at address 0x2000:



The **least significant** byte goes at the lowest address. Every x86 machine and every virtio structure works this way. When you look at a hex dump of guest memory, read multi-byte numbers right to left.

3. The fetch → decode → execute loop



- Instructions on x86 have **variable length** (1 to 15 bytes). `hlt` is 1 byte (`F4`) and `mov al, 0x48` is 2 (`B0 48`). After running one, the CPU moves RIP past it.
- **Jumps** (`jmp` , `jz` , `call` , `ret`) simply put a new value into RIP. That's how loops and functions work.
- The CPU never "knows" it's running a program. It just keeps fetching from wherever RIP points. If RIP points at garbage, it runs garbage (and usually faults).

4. Sizes you'll see everywhere

Name	Bytes	Hex	Where in kvm-hello
1 KiB	1,024	`0x400`	-
4 KiB (one **page**)	4,096	`0x1000`	each virtio-mmio window, the raw guest's load address
1 MiB	1,048,576	`0x10_0000`	where Linux's kernel is loaded
1 GiB	2^{30}	`0x4000_0000`	identity-mapped region in long mode (M2)
3 GiB	3×2^{30}	`0xC000_0000`	guest RAM cap in Phase 2
4 GiB	2^{32}	`0x1_0000_0000`	the most a 32-bit address can reach

KiB vs KB: KiB = 1024 bytes and KB = 1000. Systems code always means the 1024 kind, even when people write "KB".

5. Physical vs virtual addresses (a teaser)

So far an address is "the number of a box in RAM". That's a **physical address**. Real programs actually use **virtual addresses**, which the CPU translates through page tables (Level 1). Inside a VM there are *three* kinds of address: guest-virtual, guest-physical and host-virtual. Don't worry about that yet. Just remember that "address" will soon need an adjective.

WHY A VMM CARES

To the VMM, the guest's whole "RAM" is just one big buffer it allocated in its own process. When the guest says "physical address `0x1000`", that really means "byte `0x1000` of the VMM's buffer". The VMM copies the guest program into that buffer, sets RIP, and lets the real CPU run it.

IN THIS REPO

`src/arch/x86_64/mod.rs` is the guest's memory map, written as constants: `RAW_IMAGE_START = 0x1000` (where tiny test programs go), `BOOT_STACK_POINTER = 0x8ff0`, `CMDLINE_START = 0x2_0000`. Read that file now; every number in it is "a box on the street".

COMMON MISTAKE

Mixing up the **address** and the **value**. "Box 0x1000 contains 0xB0" has two different numbers in it. In assembly, `mov al, 0x10` loads the number 0x10, but `mov al, [0x10]` loads *whatever is in box 0x10*. The square brackets mean "go look inside".

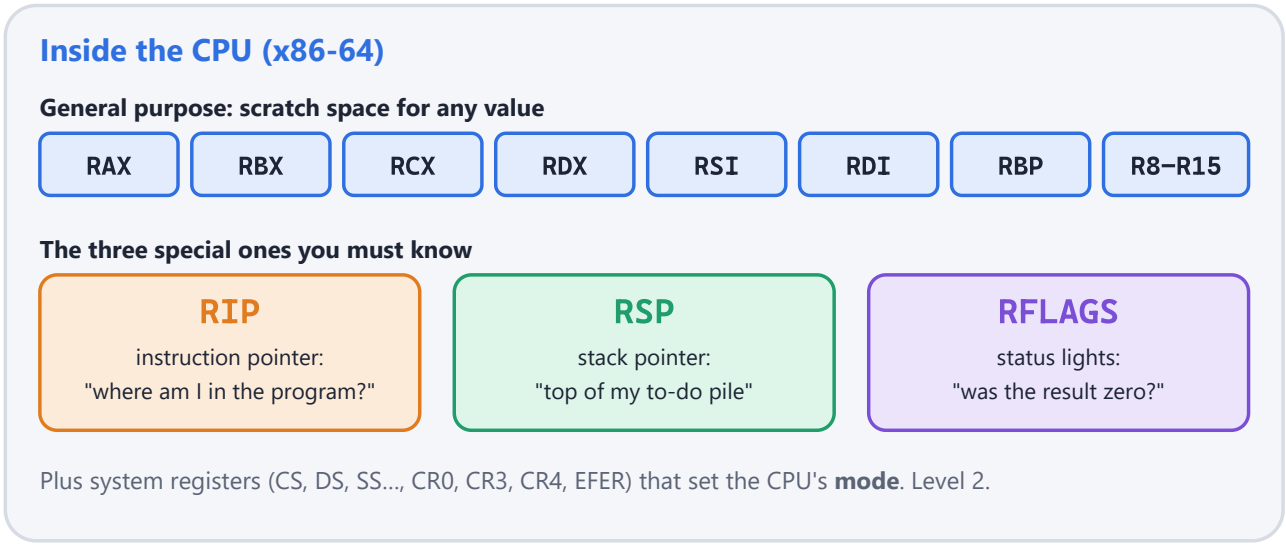
CHECK YOURSELF

1. A 2-byte value `0xBEEF` is stored at address `0x500`. Which byte is in box `0x500`, and which in `0x501`?
2. If RIP = `0x1000` and the instruction there is 2 bytes long (and not a jump), what's RIP after it runs?
3. Why can the same memory byte be both "an instruction" and "a letter of text"?
4. How many 4 KiB pages fit in 1 MiB?

0.2

Registers: the CPU's pockets

> Memory is a huge warehouse, but it's **far away and slow**. Registers are a handful of **pockets on the CPU itself**: tiny, named, and instant. The CPU does almost all of its work in its pockets.

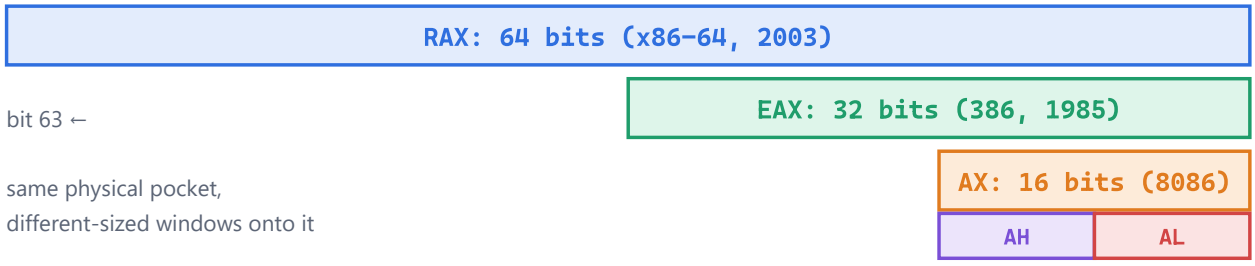


In plain words

To add two numbers in memory, the CPU first copies them into registers, adds them there, and maybe copies the answer back. Registers are the CPU's **entire current state**. If you saved every register's value, you could freeze a program and resume it later exactly where it was. **That one sentence is the key to virtualization.**

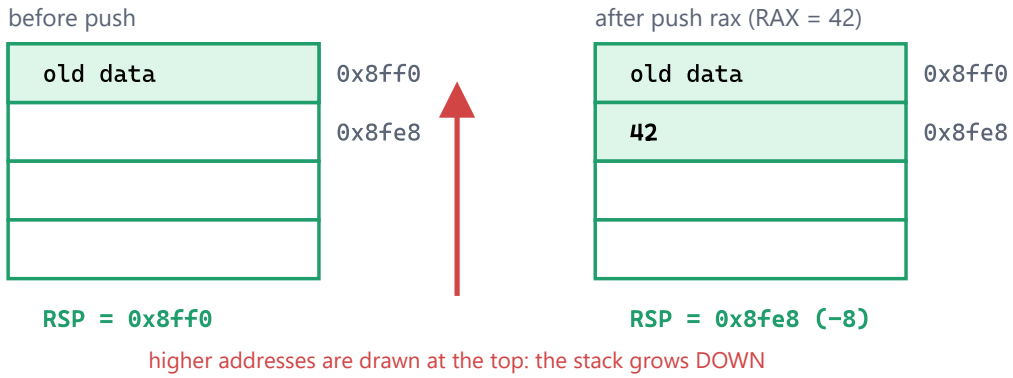
GO DEEPER

1. One register, many sizes (history in layers)



x86 has stayed backward-compatible since 1978, so each register has nested names. `AL` is the lowest 8 bits of `RAX`. Writing `AL` changes `RAX`. The same pattern applies to RBX/EBX/BX/BL, RDX/EDX/DX/DL, and so on. `hello16.asm` uses `AL` (one character) and `DX` (a port number), because in 16-bit real mode only the 16-bit names exist.

2. The stack (RSP)



- The **stack** is ordinary memory used as a pile of plates. `push` subtracts 8 from RSP and writes there. `pop` reads from RSP and adds 8.
- `call f` pushes the return address (the RIP after the call) and jumps to `f`. `ret` pops it back into RIP. That's how a function knows where to return.
- The stack **grows toward lower addresses**. That's why the VMM picks a high-ish address for the initial stack and lets it grow down into free space.

3. RFLAGS: the status lights

Bit	Name	Meaning
0	CF (carry)	unsigned overflow: the last add carried out of the top bit
1	(reserved)	**always 1**. That's why the VMM sets RFLAGS to `0x2`.
6	ZF (zero)	the last result was zero. `jz` means "jump if ZF is set".
7	SF (sign)	the last result was negative (top bit set)
9	IF (interrupt enable)	1 = interrupts may interrupt me. `cli` clears it and `sti` sets it (see 0.5).
11	OF (overflow)	signed overflow

Comparisons work in two steps. `test al, al` or `cmp` does a calculation *only to set flags*, then a conditional jump (`jz`, `jnz`, `jl` ...) looks at the flags.

WHY A VMM CARES

A vCPU (virtual CPU) is, at heart, **a saved set of register values**. Before the first run, the VMM fills in the registers. KVM's `KVM_SET_REGS` sets the general ones, and `KVM_SET_SREGS` sets the system ones. When the guest stops (a "VM exit"), the VMM can read them back. Freeze the registers plus memory and you've frozen the whole machine. That's how snapshots and live migration work.

IN THIS REPO

`src/arch/x86_64/regs.rs:48-52` is the whole idea in four lines:

```
let regs = kvm_regs {
    rip: entry,                 // start executing here (0x1000)
    rsp: BOOT_STACK_POINTER,    // stack starts at 0x8ff0, grows down
    rflags: RFLAGS_RESERVED,    // 0x2: bit 1 must be set, IF = 0
    ..Default::default()        // every other register = 0
};
```

COMMON MISTAKE

Thinking registers live in memory. They don't have addresses, so you can't point at RAX. That's also why the VMM needs special `ioctl`s to reach them: there's no memory location to just poke.

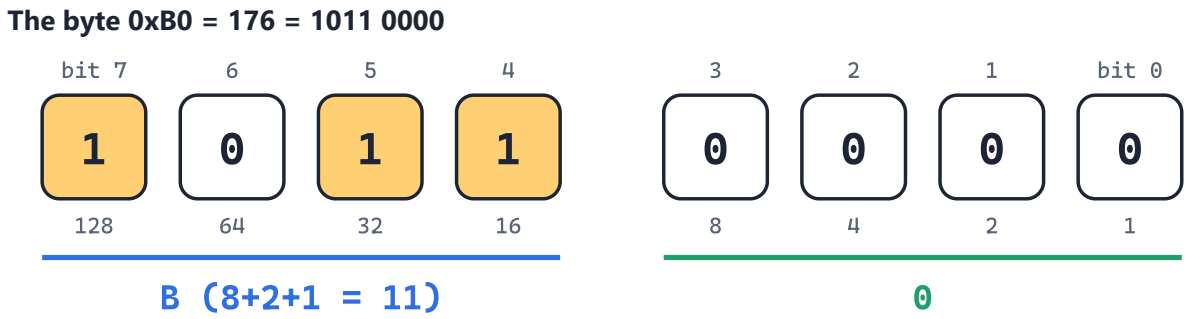
CHECK YOURSELF

1. RAX = `0x1122334455667788`. What is AL? What is AX?
2. RSP = `0x8ff0`. You do `push rax`, then `push rbx`. What is RSP now, and where is RBX's value stored?
3. Why must the VMM set RFLAGS to `0x2` instead of `0`?
4. Name the minimum things you'd need to save to "pause" a running CPU and resume it later.

0.3

Binary, hex, bits and flags

> Computers only store **on/off switches (bits)**. Hexadecimal is just a **short way to write switches**: each hex digit is exactly 4 switches. Hardware uses individual switches as **yes/no settings (flags)**.



Lit switch = 1. Add up the place values of the lit ones: 128+32+16 = 176. Split into two groups of four: 1011 = B, 0000 = 0, so the byte is 0xB0.

In plain words

Decimal counts in tens because we have ten fingers. Computers count in twos. Binary gets long fast, so programmers use **hex** (base 16, digits `0–9` then `A–F`). Hex is popular because **1 hex digit = 4 bits exactly**, so it converts to and from binary in your head. Two hex digits make one byte.

GO DEEPER

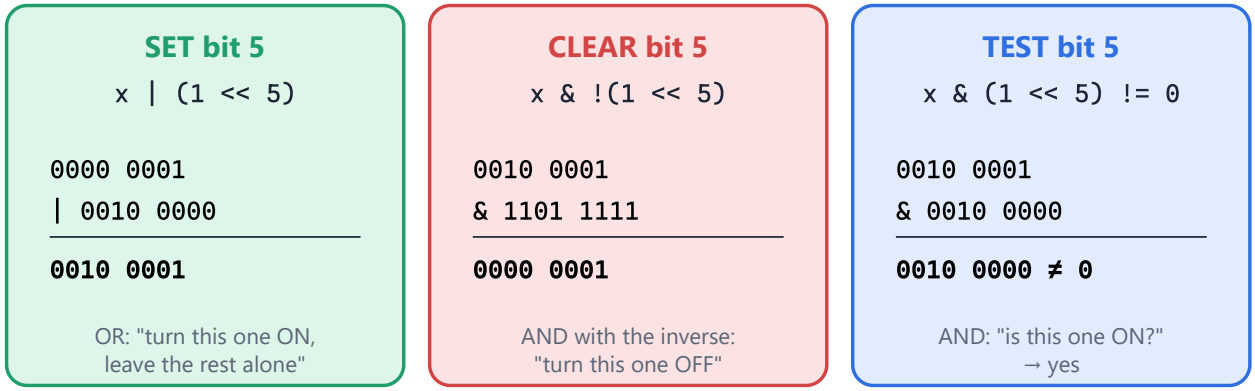
1. The conversion table to memorise

Hex	Binary	Dec		Hex	Binary	Dec
0	0000	0		8	1000	8
1	0001	1		9	1001	9
2	0010	2		A	1010	10
3	0011	3		B	1011	11
4	0100	4		C	1100	12
5	0101	5		D	1101	13
6	0110	6		E	1110	14
7	0111	7		F	1111	15

- **Prefixes:** `0x` means hex and `0b` means binary. Rust lets you write `0xd000_0000` with underscores for readability, and the underscores mean nothing.
- **Reading big hex fast:** every 4 hex digits is a factor of 65,536. `0x1000` = 4096 = 4 KiB. Each extra zero multiplies by 16.
- **Powers of two to know:** 2^{10} = 1 Ki, 2^{12} = 4 Ki (a page), 2^{16} = 64 Ki (the number of I/O ports), 2^{20} = 1 Mi, 2^{30} = 1 Gi, 2^{32} = 4 Gi.

2. Flags: one bit = one yes/no setting

Hardware packs many on/off settings into one number. "Turn on paging" really means "set bit 31 of register CR0". Three operations do all the work:



`1 << n` ("one shifted left by n") is a number with only bit *n* lit. It's called a **mask**. Note that Rust writes NOT as `!`, while C writes it as `~`.

3. Signed numbers and overflow (two's complement)

- The same 8 bits `1111 1111` mean **255** if you treat them as unsigned (`u8`) and **−1** if you treat them as signed (`i8`). The bits don't change. Only the interpretation does.
- Fixed-size numbers **wrap around** like a car odometer: `255u8 + 1` becomes `0`. In a debug build Rust panics on this by default.
- That matters in this project. A guest can hand the VMM *any* number, for example an address of `0xFFFF_FFFF_FFFF_FFF0` with a length of `0x20`. A naive `addr + len` overflows. The VMM uses `checked_add` so a malicious guest can't crash it. You'll meet this again in Level 7.

IN THIS REPO

`src/arch/x86_64/regs.rs:11-15` is all flag masks:

```
const CR0_PE:   u64 = 1 << 0;   // Protection Enable
const CR0_PG:   u64 = 1 << 31;  // PaGing
const CR4_PAE:  u64 = 1 << 5;   // Physical Address Extension
const EFER_LME: u64 = 1 << 8;   // Long Mode Enable
const EFER_LMA: u64 = 1 << 10;  // Long Mode Active
```

Later the code does `sregs.cr0 |= CR0_PE | CR0_PG;`, which means "switch on both settings". You now know exactly what that line does.

COMMON MISTAKE

Off-by-one bit numbering. Bits are numbered from **0**, starting at the right (least significant). "Bit 5" is the *sixth* switch from the right and has value 32 (`0x20`).

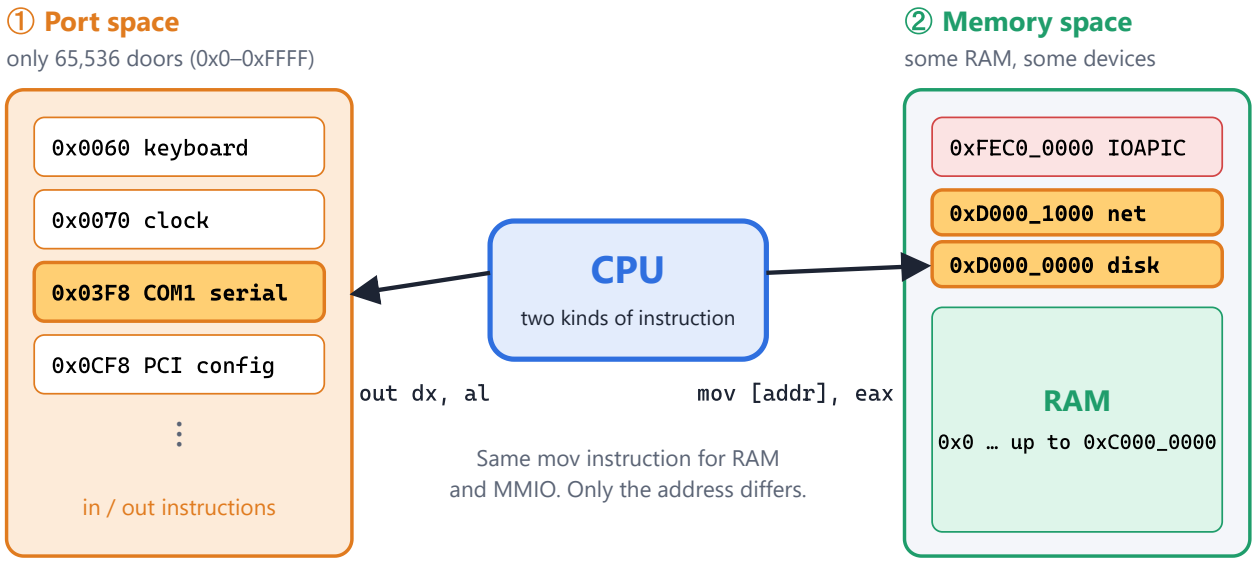
CHECK YOURSELF

1. Convert `0x3F8` to binary and to decimal.
2. What's `1 << 31` in hex?
3. x = `0b1010_1010` . Write the result of setting bit 0 and clearing bit 7.
4. How many bytes is `0xd000_1000 - 0xd000_0000` ? What's it called?
5. Why is `addr + len` dangerous when the guest chooses both numbers?

0.4

Talking to devices: I/O ports vs MMIO

> A CPU can only read and write numbers. So devices are made to **look like places you read and write**. x86 offers two kinds of "places":
> ① a separate small set of **numbered doors (I/O ports)**, and
> ② **fake memory addresses (MMIO)** that are wired to a device instead of RAM.



The orange boxes are exactly the devices kvm-hello fakes. The addresses on the right are the real Phase 2 memory map.

In plain words

A device has a few **registers** of its own, little mailboxes such as "byte to send", "status" and "command". To use the device, the CPU reads or writes those mailboxes. The only question is how they're addressed: by **port number**, using the special `in` / `out` instructions, or by **memory address**, using ordinary `mov`.

GO DEEPER

1. Port I/O (PIO)

- A **separate, tiny address space** of 2^{16} ports, unrelated to RAM. Port `0x3F8` and memory address `0x3F8` are totally different things.
- Only two instructions use it: `out dx, al` ("send byte AL to port DX") and `in al, dx` ("read a byte from port DX into AL").

- It's x86-specific and old (from the 1970s). It survives for legacy PC devices: the serial port, keyboard controller, timer and PCI config.
- A device usually takes a **small range** of ports. COM1 uses 8 of them, `0x3F8–0x3FF`, and each one is a different UART register.

COM1 port	Writing it means...	Reading it means...
`0x3F8` (+0)	transmit this byte	receive a byte
`0x3F9` (+1)	which interrupts to enable	same
`0x3FD` (+5)	-	line status: "ready to send?" / "byte waiting?"

2. Memory-mapped I/O (MMIO)

- Some ranges of the **memory** address space are wired to a device instead of RAM. Reading `0xD000_0000` doesn't fetch from a memory chip. It asks the disk device "what's your magic value?"
- It uses normal instructions, so it works on every architecture (ARM and RISC-V have no ports at all). There's room for many devices, and a large register window.
- Device registers sit at an **offset** inside the window. For virtio-mmio, offset `0x000` is MagicValue ("virt"), `0x070` is Status, and so on (Level 4).
- The hole must **not overlap RAM**. That's why the spec caps guest RAM at 3 GiB, so it ends at `0xC000_0000` and leaves `0xD000_0000` free for devices.

3. Side effects: why devices aren't memory

Reading RAM twice gives the same answer. Reading a device register can **change things**: reading the UART's receive register *removes* that byte from its queue. Writing can *do* things, like sending a character or starting a disk read. That's why device registers can't be cached like RAM, and why every access matters.

WHY A VMM CARES

This is the heart of a VMM. In a VM **there is no real serial chip or disk at those addresses**. When the guest runs `out dx, al` to port 0x3F8, the real CPU stops the guest and tells KVM, which tells our VMM: `KVM_EXIT_IO { port: 0x3f8, data: 'H' }`. Our VMM pretends to be the UART and prints 'H'. MMIO works the same way: touching `0xD000_0000` (deliberately left with no RAM behind it) produces `KVM_EXIT_MMIO`. This is called **trap-and-emulate** (Level 3).

IN THIS REPO

`src/devices/serial.rs:20` has `COM1_BASE: u16 = 0x3f8`. Note the `u16`: ports are 16-bit. `src/devices/pio_bus.rs` routes a port number to a device, and `src/devices/mmio_bus.rs` routes a memory address to a device.

`src/arch/x86_64/mod.rs:34-42` defines `BLK_MMIO_BASE = 0xd000_0000` and `NET_MMIO_BASE = BLK + 0x1000`.

`src/lib.rs:183` tells the Linux guest where the devices are, through the kernel command line: `virtio_mmio.device=4K@0xd0000000:5` (size, address, IRQ).

COMMON MISTAKE

Thinking port 0x3F8 is "memory address 0x3F8". They're different spaces, chosen by the instruction (`out` vs `mov`), not by the number.

CHECK YOURSELF

1. Why does x86 need two different instructions (`out` and `mov`) to reach devices, while ARM needs only one?
2. Why would putting virtio-blk's MMIO window at `0x1000_0000` be a bug when the guest has 1 GiB of RAM?
3. Reading a UART register twice gave two different answers. Is that a bug? Why or why not?
4. In a VM, what makes the real CPU stop and hand control to the VMM when the guest writes to port 0x3F8?

0.5

Interrupts

> Instead of the CPU asking the keyboard "any key yet? any key yet?" a million times a second (**polling**), the keyboard **taps the CPU on the shoulder** when a key arrives. That tap is an **interrupt**.

Polling 😩

CPU: "Data yet?" Device: "No."
CPU: "Data yet?" Device: "No."
CPU: "Data yet?" Device: "No."
CPU: "Data yet?" Device: "Yes!"

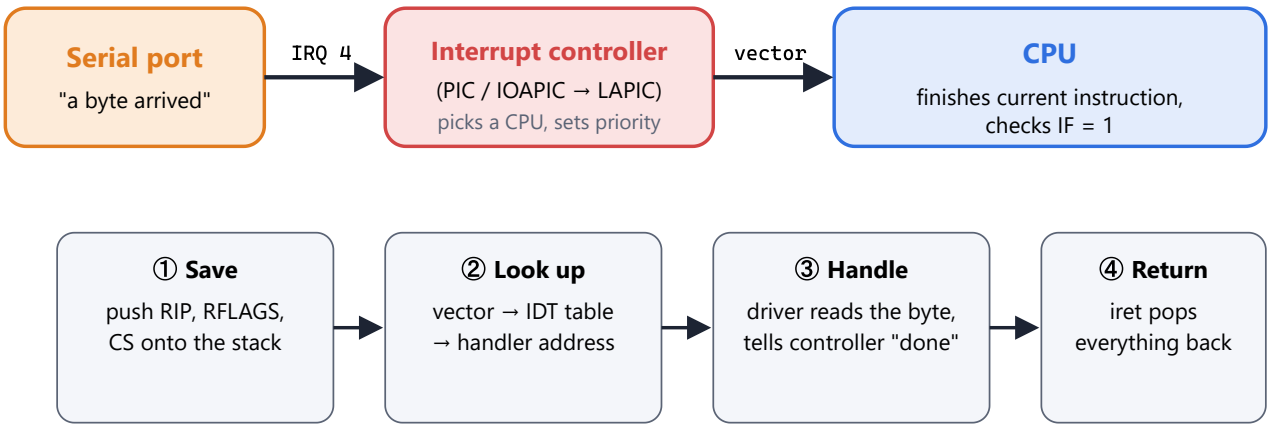
wastes the CPU while it waits

Interrupts 😌

CPU: does useful work... or sleeps (hlt)
CPU: ...
Device: *tap* "IRQ 4: data ready!"
CPU: pauses, handles it, resumes

the CPU is only bothered when needed

The journey of one interrupt



The interrupted program never notices. It resumes exactly where it was, because its registers were saved.

In plain words

Each device gets a **line number** (IRQ, "interrupt request"). When it needs attention, it raises its line. An **interrupt controller** collects all the lines and passes the request to a CPU. The CPU pauses what it's doing, runs a small piece of code for that interrupt (the **handler**), then resumes. All of this is done by hardware, without the running program asking for it.

GO DEEPER

1. Vocabulary

Term	Meaning
IRQ / GSI	the device's interrupt line number. In kvm-hello: COM1 = 4, virtio-blk = 5, virtio-net = 6. KVM calls the global numbering "GSI".
Interrupt controller	the switchboard. Old PCs had the **PIC** (8259). Modern ones have an **IOAPIC** (collects device lines) plus a **LAPIC** inside each CPU core (receives them). Level 4 and Level 5 cover these.
Vector	a number 0–255 telling the CPU *which* handler to run.
IDT	the Interrupt Descriptor Table: an array in memory mapping vector → handler address. The OS fills it in.
Handler / ISR	the code that services the device: reads the byte, completes the disk request, and so on.
EOI	"end of interrupt": the handler tells the controller it's done, so more can come.
Edge vs level	edge = "a pulse happened", level = "the line stays high until serviced". kvm-hello's virtio interrupts are edge-triggered.

2. Three kinds of "interruption"

- **Hardware interrupts**: from devices (this chapter).
- **Exceptions**: caused by the instruction itself, such as divide by zero (vector 0), page fault (vector 14) or general protection fault (vector 13). They use the same IDT mechanism.
- **Software interrupts**: the program asks on purpose (`int 0x80`). This is the old way to make a system call (Level 1).

3. Turning interrupts off: `cli` , `sti` , `hlt`

- `cli` clears RFLAGS.IF, which means "don't interrupt me". `sti` sets it again. The kernel does this around delicate code.
- `hlt` means "sleep until the next interrupt". It's how an idle CPU saves power. Linux's idle loop is basically `sti; hlt` forever.
- `cli; hlt` together means "sleep, and nobody may wake me": a permanent stop. That's exactly how `hello16.asm` ends. In raw mode (no in-kernel interrupt controller), `hlt` makes the CPU exit to the VMM (`KVM_EXIT_HLT`), and the VMM treats that as "the guest is done, exit 0".

WHY A VMM CARES

In a VM, devices are fake, so interrupts are fake too. The VMM must **inject** them. kvm-hello asks KVM to emulate the interrupt controllers inside the kernel (`setup_irqchip` , "irqchip" = interrupt controller). It then hands KVM an **eventfd** per device through `KVM_IRQFD` . When the fake disk finishes a request, the VMM writes to that eventfd, and KVM delivers IRQ 5 into the guest. The guest's Linux driver can't tell the difference from real hardware.

IN THIS REPO

`src/devices/serial.rs:24` : `COM1_IRQ: u32 = 4` .

`src/lib.rs:235-236` : `vm.setup_irqchip()` , which creates the virtual PIC, IOAPIC and LAPICs. It must happen before the vCPUs are created.

`src/lib.rs:290-292` : `register_irqfd(&irq_evt, COM1_IRQ)` , "writing to this eventfd = raise IRQ 4".

`src/arch/x86_64/regs.rs:17` : RFLAGS starts at `0x2` , so **IF = 0**. The guest starts with interrupts off and turns them on itself once its IDT is ready.

COMMON MISTAKE

Thinking an interrupt runs "in parallel". It doesn't. The same CPU stops the current program between two instructions, runs the handler, then resumes. Concurrency bugs still happen, though, because the handler and the program share data. That's why kernels use `cli` .

CHECK YOURSELF

1. Why is polling wasteful, and when might it actually be better? (Hint: think "very fast device, very busy CPU".)
2. What happens to the interrupted program's RIP while the handler runs? How does it get back?
3. Why does the guest start with IF = 0 instead of 1?
4. What's the difference between `hlt` and `cli; hlt` ?
5. In a VM there's no wire from a fake disk to the CPU. How does kvm-hello deliver IRQ 5?

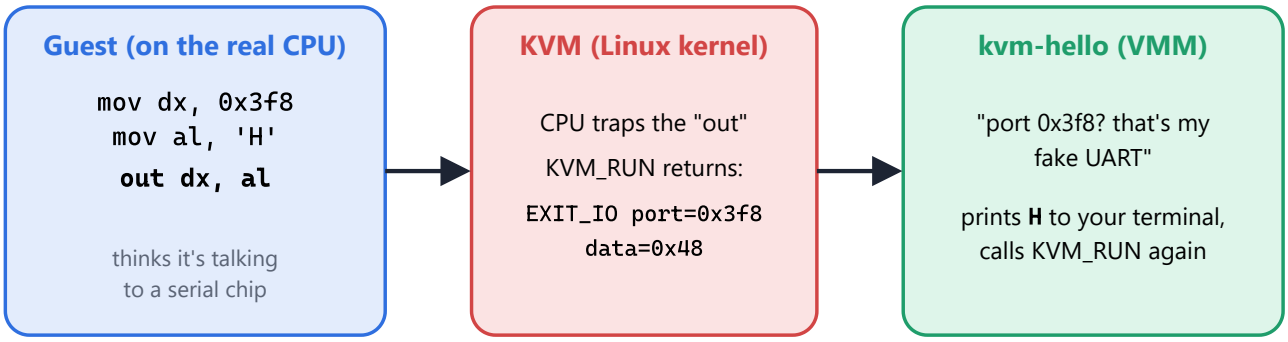
All five ideas in one tiny program

`guests/hello16.asm` is the very first guest this project ever ran (milestone M1). It uses every Level 0 concept. Read it slowly. The [0.x] tag says which section each line uses.

```
; loaded at guest-physical 0x1000    [0.1]
bits 16
org 0x1000

start:
    cld             ; clear Direction Flag (RFLAGS bit)  [0.3]
    mov dx, 0x3f8   ; DX = COM1 transmit port            [0.2, 0.4]
    mov si, msg     ; SI = address of the string         [0.1]
.next:
    lodsb           ; AL = byte at [SI]; SI += 1         [0.1]
    test al, al     ; AND AL with itself to set ZF       [0.2]
    jz .done        ; ZF=1 (byte was 0)? RIP = .done     [0.1]
    out dx, al      ; AL to port DX: KVM_EXIT_IO         [0.4]
    jmp .next
.done:
    cli             ; IF = 0: no interrupts can wake us  [0.5]
    hlt             ; sleep: KVM_EXIT_HLT (raw mode)     [0.5]

msg: db "Hello from real mode!", 10, 0   ; ASCII, newline, NUL
```



This loop (run the guest → it exits → the VMM emulates the device → run again) is the whole job of a VMM. Everything in Levels 1–7 is detail on top of this picture.

Hands-on exercises (do these after reading)

1. **Read the bytes.** Run `xxd guests/build/hello16.bin | head`. Find the bytes `BA F8 03`: that's `mov dx, 0x3f8`. Notice `F8 03` is 0x03F8 stored little-endian. Then find `EE` (`out dx, al`) and `F4` (`hlt`).
2. **Find the string.** In the same dump, find `48 65 6C 6C 6F`, which is "Hello" in ASCII.

3. **Map the memory.** On paper, draw guest memory from `0x0` to `0x2_1000` using the constants in `src/arch/x86_64/mod.rs`: the GDT, the raw image, boot params, the stack, the page tables and the command line. Check that nothing overlaps.
4. **Bit practice.** Work out by hand `CR0_PE | CR0_PG` in hex. Then work out `0x8000_0011 & CR0_PG`, and say whether it's zero.
5. **Predict, then check.** If you deleted the `cli` before `hlt`, would this guest behave differently in raw mode? (Hint: what wakes a halted CPU, and does raw mode have an interrupt controller?)

Cheat sheet

Concept	One-line memory hook
Memory	A street of numbered boxes, one byte each. The address is the box number.
Little-endian	The small end goes first: 0x1234 is stored as `34 12`.
CPU loop	Fetch at RIP → decode → execute → advance. Forever.
Registers	The CPU's pockets. Saving them all freezes the program.
RIP / RSP / RFLAGS	Where am I / top of the stack / status lights.
Stack	Grows down. push: RSP −= 8. call pushes the return address.
Hex	1 hex digit = 4 bits. 0x1000 = 4 KiB = one page.
Flags / masks	Set `|=`, clear `&= !`, test `& != 0`. Mask = `1 << n`.
Overflow	Fixed-size numbers wrap. Guest-chosen numbers need `checked_add`.
Port I/O	65,536 separate doors, using `in` / `out`. COM1 = 0x3F8.
MMIO	Device registers at "fake RAM" addresses, using normal `mov`. virtio at 0xD000_0000.
Side effects	Device registers act when touched. Reading can consume data.
Interrupt	A device taps the CPU: save → IDT lookup → handler → iret.
IRQ	The device's line number: COM1 4, blk 5, net 6.
IF, cli/sti, hlt	Allow interrupts / block / allow / sleep until one arrives.
VMM's trick	Fake devices trap out (EXIT_IO / EXIT_MMIO). Fake interrupts are injected (irqfd).

Answers

0.1 CPU, memory and addresses

1. `0x500` holds `EF` and `0x501` holds `BE` (little-endian).
2. `0x1002`.
3. Memory just holds numbers. Meaning comes from use: if RIP points there, it's code; if a `mov` reads it, it's data.
4. 1 MiB / 4 KiB = 256 pages.

0.2 Registers

1. AL = `0x88` and AX = `0x7788`.
2. RSP = `0x8fe0`. RBX's value is at `0x8fe0`, and RAX's is at `0x8fe8`.
3. Bit 1 of RFLAGS is architecturally reserved and must be 1. The hardware's VM-entry checks refuse to start a guest whose RFLAGS breaks this rule.
4. All registers (general, RIP, RSP, RFLAGS, plus segment and control registers) and memory. Device state matters too, which comes later.

0.3 Binary, hex, bits and flags

1. `0x3F8` = `0011 1111 1000` = 1016.
2. `0x8000_0000`.
3. `0b0010_1011`.
4. `0x1000` = 4096 bytes, one 4 KiB page.
5. The sum can wrap around to a small number, which passes a bounds check while pointing somewhere else. Or, in debug builds, it panics the VMM.

0.4 I/O ports vs MMIO

1. x86 has a separate port address space for historical reasons. ARM only has memory space, so all devices are MMIO.
2. It falls inside guest RAM (0 to 0x4000_0000), so the guest would see RAM there and the VMM would never get an exit.
3. Not a bug. Device reads have side effects; reading the receive register pops a byte.
4. The CPU's virtualization hardware is configured to trap I/O instructions. KVM then returns `KVM_EXIT_IO` to the VMM.

0.5 Interrupts

1. Polling burns CPU while it waits. It can win when events come extremely often (high-speed networking busy-polls) because it avoids the per-interrupt overhead.
2. It's pushed on the stack. `iret` pops it back into RIP.
3. The guest has no IDT yet. An interrupt arriving now would jump to a garbage handler.
4. `hlt` sleeps until an interrupt. `cli; hlt` sleeps with interrupts blocked, so effectively forever (only special NMI/reset events wake it).
5. It registers an eventfd with `KVM_IRQFD` for GSI 5. Writing to the eventfd makes KVM's in-kernel interrupt controller raise IRQ 5 in the guest.

Next: Level 1, the operating system. Covers user vs kernel mode, system calls, file descriptors, `ioctl`, virtual memory, `mmap`, threads, epoll/eventfd, signals and terminals.